

A Toolkit for Empirical Impulse Response Preparation

Victor Lazzarini¹

Music Department
Maynooth University
Ireland
`victor.lazzarini@mu.ie`

Abstract. Empirical impulse responses can be obtained by measuring physical systems. Csound has built-in means for utilising these in musical applications through its various types of convolution opcodes. Until now, the system lacked out-of-the-box means for preparing empirical impulse responses, although this can actually be implemented from first principles in Csound code. This paper introduces a native toolkit developed for this purpose, in three forms: a utility, a GEN, and opcodes.

Keywords: impulse response; convolution; spectral processing;

1 Introduction

Csound [2] includes a number of opcodes that utilise impulse responses (IRs). The first one of these, introduced in an early version, is `convolve`, which implements overlap-add single-partition convolution using an analysis file produced from an IR by the `cvanal` utility. Later on, multiple-partition convolution was supported by the `pconvolve` and `ftconv` opcodes using overlap-save and overlap-add methods, respectively. The former takes IRs from audio files, and the later reads them from function tables. Since these are all based on frequency-domain methods, which introduce a varying amount of input-output latency, Csound also includes a time-domain convolution opcode `dconv` can be used for zero-delay short IR. In particular, this is very useful to handle the lead partition in multiple-size partitioned convolution in conjunction with `ftconv`. Finally, we have `liveconv`, which allows IRs to be swapped seamlessly, implementing a form of time-varying convolution. Besides these, Csound has support for implementing all methods from first principles.

The creation of IRs can be done via synthetic or empirical methods. The former employs different mathematical models for creating IRs for specific effects. For example, it is possible to create artificial reverb IRs from a given mathematical model of a space. An IR can also be created from a given frequency response using GEN53. The empirical approach, alternatively, involves *measuring* an IR from an existing system, which may be for instance an electrical circuit, an electromechanical device, or a physical space/object. Csound does not provide an out-of-the-box solution for preparing these, besides the trivial generation of an impulse signal, which has limited use (it can be employed to obtain an IR from a mathematical model through sampling). However, code can be used to implement all the necessary operations from first principles.

In this paper, a solution for preparing empirical IRs is discussed. It involves the implementation of the necessary operations to produce and process signals for measuring a system IR. We first discuss the technical details of the capture method. Then we proceed to show a prototype implementation in Csound. Finally, we discuss the toolkit and its use.

2 Impulse Responses

An impulse response is the output of a linear-time invariant (LTI) system when presented with an impulse as an input. In discrete-time, an impulse is a signal consisting of a unit sample followed by zeros,

$$u(n) = \{1, 0, 0, \dots\}. \quad (1)$$

The LTI system can be modelled by a finite impulse response filter of order K ,

$$y(n) = \sum_{k=0}^K a_k x(n-k), \quad (2)$$

in which its impulse response is equivalent to the coefficients a_k .

The filter output therefore is the convolution of two signals, the impulse response $h(n)$ and the filter input $x(n)$,

$$y(n) = \sum_{m=0}^{M-1} h(m)x(n-m) = h(n) \otimes x(n). \quad (3)$$

Moreover, the spectrum of the convolution $Y(z)$ is given by the product of the two input spectra, $H(z)$ and $X(z)$ [4],

$$Y(z) = H(z)X(z). \quad (4)$$

For a given LTI system, we may obtain an IR through an empirical approach. Given an impulse as an input signal, the measured output yields an estimate of the system IR (Fig. 1).

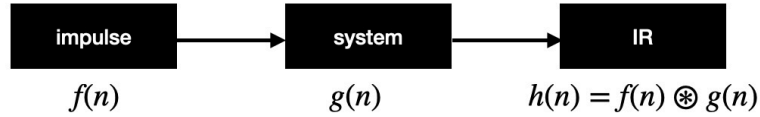


Fig. 1. Empirical IR measurement.

Ideally, $h(n) \approx g(n)$ but that will depend on the conditions of measurement; if $f(n) = u(n)$ and the capture is linear/lossless, then $h(n) = g(n)$.

3 Sine Sweep Method

A typical method is to use a sine sweep signal covering the full spectrum [1], which effectively smears the impulse in time (Fig. 2).

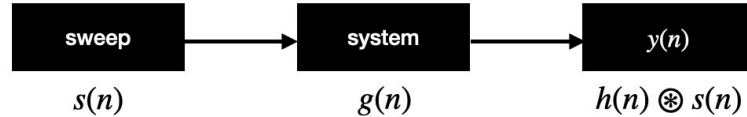


Fig. 2. Sine sweep IR capture.

Assuming a linear/lossless capture, the result is the convolution of the sweep and the system impulse response. We can recover $h(n)$ by applying the inverse convolution of $y(n)$ by $s(n)$. Following the relationships given earlier, a common approach to compute this is to take the spectral quotient of the captured output and the sweep signal $y(n)$ and the sweep signal (Fig. 3). For this purpose, $Y(z)$ and $S(z)$ are produced by applying the discrete Fourier transform (DFT) of $y(n)$ and $s(n)$, respectively, normally computed using the fast Fourier transform (FFT). The IR, $h(n)$, is obtained by taking the inverse DFT of $Y(z)/S(z)$.

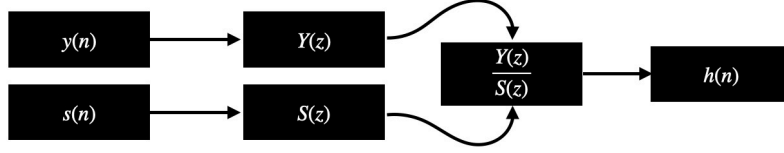


Fig. 3. Inverse convolution through the spectral method.

The sweep size determines the length l of the empirical IR measurement. The size of the convolution output is exactly $2l - 1$ samples. We should then match the DFT size to accommodate this; the sweep signal $s(n)$ is padded with zeros to this length. The measured IR as the inverse convolution output, $h(n)$, is thus l samples long. The remaining samples from the IDFT output are discarded.

4 Prototype

We generate a sweep with an exponential frequency function covering the full spectrum for the set duration of the IR, creating the waveform depicted in Fig. 4.

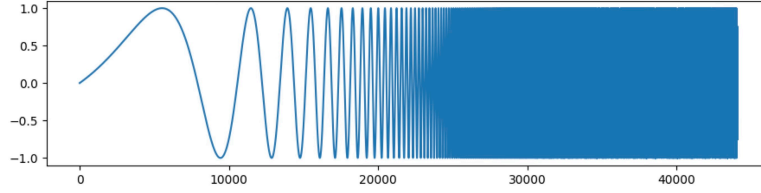


Fig. 4. Sweep signal waveform.

In Csound, this can be created with the single-line instrument

```
out(oscili(1, expon(1,p3,sr/2)))
```

scheduled for the required duration.

Next, we need to be able to apply the inverse convolution to the captured measurement. With a padded sweep, we get the product of its inverse spectrum and the input spectrum, and apply the IDFT. In Csound 7, with its complex number support [3], the code is simply

```
instr mkir
  swfn:i = ftgen(1,0,0,-1,"sweep.wav",0,0,1)
  infn:i = ftgen(2,0,0,-1,"capture.wav",0,0,1)
  irfn:i = ftgen(3,0,ftlen(swfn),2,0)
  swp:k[] = init(ftlen(swfn)*2)
  inp:k[] = init(ftlen(infn))
  copyf2array(swp,swfn)
  copyf2array(inp,infn)
  sws:Complex[] = rfft(swp)
  spc:Complex[] = rfft(inp)
  ir:k[] = rfft(spc/sws)
  copya2ftab(ir,irfn)
  turnoff
endin
```

with the k-arrays `swp` and `inp` holding the sweep and input, each being twice the length of the sweep signal. Here we can use real-to-complex FFT and complex-to-real inverse FFT. The `swp` array is one sample longer than the convolution size, which is required in this case to make the `rfft` input even, avoiding an init error in Csound without any material influence on the result. Figures 5 and 6 show the captured signal used as input to the inverse convolution and the resulting empirical IR, respectively.

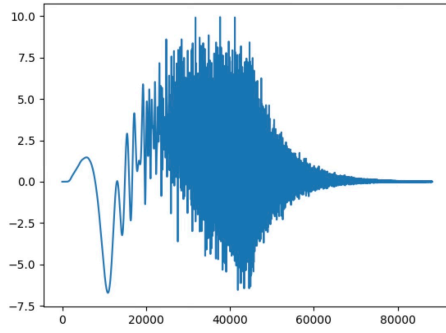


Fig. 5. Captured signal.

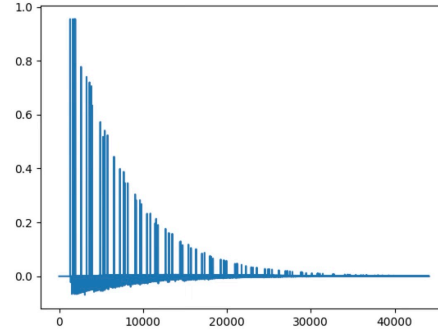


Fig. 6. Measured empirical IR.

5 The IR Preparation Toolkit

The toolkit supports IR preparation with Csound in three different forms:

1. Standard utility

- Sweep generation,

```
mkir -g output_file duration
```

- IR recovery,

```
mkir sweep_file -i input_file -o output_file
```

2. GEN routine

The function table generator takes sweep data and input(s) from other function tables. It can create interleaved multichannel IR tables if given more than one input,

```
ftnum:i = ftgen(0,0,0,"deconv",sweepfn:i,inputfn:i[,inputfn2:i,...])
```

The generated table is ready to be used with `ftconv`, `liveconv`, or `dconv`.

3. Opcodes

These take inputs as arrays and return an array as output,

```
ir:i[] = deconv(input:i[],sweep:i[])
ir:k[] = deconv(input:k[],sweep:k[])
```

They produce the inverse convolution of their first input by the second. The output array is the same length l as the second input. The first input can be of any length, but only the first $2l - 1$ samples are used.

All three share the same core processing code. Depending on the application, one of them is the more convenient tool. For example, convolution opcodes that use files as input are best served by the utility. For those that employ function tables, the GEN routine is a good choice. For a more general use of inverse convolution, the opcode interface is more flexible.

As an example, we can take a sweep audio file, play it through the system we want to measure, and capture the signal in a file. Following this, we load these two in tables, run the inverse convolution, overwrite the sweep table, and apply the convolution:

```
swfn@global:i = ftgen(1,0,0,-1,"sweep.wav",0,0,1)
infn@global:i = ftgen(2,0,0,-1,"input.wav",0,0,1)

instr reverb
  swp:i[] = init(ftlen(swfn))
  inp:i[] = init(ftlen(infn))
  copyf2array(swp,swfn)
```

```

copyf2array(inp, infn)
outp:i[] = deconv(inp, swp)
copya2ftab(outp, swfn)
sig:a = diskln2("fox.wav")
rev:a = ftconv(sig, swfn, 64)
        out(rev*0.5)
endin

```

5.1 Testing

How do we know we have a correct result? We can pass the sweep file through an opcode such as `reverb`, and apply the process described here to obtain the IR. Then we compare the outputs of `reverb` and `ftconv` loaded with the prepared IR, normalising the gain and compensating for delay. If the subtracted result is zero or nearly zero, we asserted that the computation was correct, within a small scaling factor.

```

swfn@global:i = ftgen(0,0,0,1,"sweep.wav",0,0,1)
instr mkrev
  aph = linseg(0,1,1,0,0)
  asig = tablei(aph,swfn,1)
  arev = reverb(asig,1)
  fout("reverb.wav",6,arev)
endin
schedule(mkrev,0,2)

instr test_ir
  scal:i = 0.95477
  inpfn:i = ftgen(0,0,0,1,"reverb.wav",0,0,1)
  irfn:i = ftgen(0,0,0,"deconv",swfn,inpfn)
  sig:a = mpulse(1,1)
  out1:a = ftconv(sig,irfn,64)
  out2:a = reverb(delay(sig, 64/sr),1)
  err:k = rms(out2 - out1*scal)
  if err > 0.00001 then
    printks("error rms: %f\n", 1, err)
  endif
  out(out1,out2)
endin
schedule(test_ir,2,1)

```

Confirming this, the code runs without printing any errors.

6 Conclusions

Csound has complete support for the use of impulse responses through the `convolve`, `pconvolve`, `dconv`, `ftconv`, and `liveconv` opcodes. Support for empirical IR preparation was not present out of the box. As we demonstrated in our prototype, Csound code can of course be written from first principles to generate the sweep signal and to process a recording through inverse convolution. This toolkit however provides a simpler alternative, implementing sweep generation and inverse convolution in three convenient forms: as a standard utility, an opcode, and a function table generator routine. They are provided to Csound in the standard utility plugin module.

References

1. Farina, S., "Simultaneous measurement of impulse response and distortion with a swept-sine technique", 108th AES Convention, Paris (February 2000)
2. Lazzarini, V., Ffitch, J., Yi, S., Heintz, J., Brandtsegg, Ø., and McCurdy, I.: Csound: A Sound and Music Computing System. Springer (2016)
3. Lazzarini, V., Yi S., Heintz, and Walsh, R.: Csound: A Sound and Music Computing System, Volume 2. Springer (2026)
4. Lazzarini, V.: Spectral Music Design: A Computational Approach. Oxford University Press (2021)